מימוש מערכת חישוב דינמית/ דוח מסכם

הסבר טכני: מימוש מערכת חישוב דינמית ב#C

המימוש ב-C# תוכנן כמנוע בעל ביצועי קצה (High Performance Engine). המערכת משתמשת ב-**Roslyn API** כדי לבצע קומפילציה של נוסחאות טקסטואליות לקוד מכונה (IL) בזמן ריצה, מה שמבטל את התקורה (Overhead) של פרשנות טקסט חוזרת. לניצול מרבי של חומרת השרת, הוטמע עיבוד מקבילי מסיבי באמצעות **Parallel.For.**

פריצת הדרך בביצועים: הזרמת הנתונים חזרה לבסיס הנתונים מבוצעת באמצעות **SqlBulkCopy** המשולב עם **Custom IDataReader**. ארכיטקטורה זו מאפשרת הזרקת מיליוני רשומות ישירות לדפי הנתונים ב-SQL Server בגישת Zero-copy, תוך חיסכון של מאות מגה-בייטים בזיכרון ה-RAM שנצרכים בדרך כלל על ידי אובייקטי **DataTable**.

1. קומפילציה בזמן ריצה(Roslyn API)

בניגוד לשיטות של "פענוח טקסט (**Parsing**) "או פרשנות (**Interpreter**) שהן איטיות משמעותית, הקוד משתמש במנוע ה Compiler-המובנה של דוט-נט-**Roslyn**

- **התהליך:** המערכת מקבלת את הנוסחה כטקסט מה ,DB-מבצעת התאמות תחביריות ומקמפלת אותה בזמן אמת לקוד מכונה.(Native IL)
- **היתרון**: לאחר הקומפילציה, הרצת הנוסחה על מיליון השורות מתבצעת במהירות של פונקציה מובנית במערכת, ללא תקורה (Overhead) של ניתוח טקסט חוזר.

2. ניהול זיכרון ועיבוד מקבילי מסיבי(Parallel Processing)

כדי למנוע צווארי בקבוק ולנצל את מלוא כוח העיבוד של השרת, נעשה שימוש בספריית **TPL** (**Task Parallel Library**).

- **אופטימיזציית** : I/O המערכת טוענת את כל מיליון הרשומות לתוך רשימה מנהלת (List)בזיכרון ה RAM-בבת אחת . פעולה זו מבטלת את צוואר הבקבוק של תקשורת הנתונים ומאפשרת למעבד גישה מיידית למידע.
- **עיבוד מקבילי** : שימוש בפקודת **Parallel.For** מאפשר לחלק את עומס החישוב בין כל ליבות המעבד (CPU Cores) בו-זמנית.
- **היתרון :** צמצום דרמטי של זמן החישוב (כפי שמוצג בתוצאות, החישוב לוקח אלפיות השנייה) בהשוואה ללולאה טורית הרצה על ליבה בודדת.

3. הזרקת נתונים אופטימלית (SqlBulkCopy & Streaming)

שלב הכתיבה לבסיס הנתונים הוא לרוב האיטי ביותר. פתרתי זאת באמצעות טכניקת Streaming מתקדמת:

- **התהליך:** במקום להשתמש באובייקטים כבדים כמו **DataTable** (שעלולים לצרוך כ-MB400 זיכרון עבור מיליון שורות), יצרתי מחלקה ייעודית בשם **BulkDataReader** הממשת את הממשק **IDataReader**.
- **היתרון:** שיטה זו משמשת כ"צינור" המזרים את התוצאות ישירות מהזיכרון ל- SQL Server בבת אחת. השימוש ב-**SqlBulkCopy** מאפשר כתיבה של מאות אלפי שורות

בשנייה במינימום צריכת RAM, תוך עקיפת מנגנוני הרישום האיטיים של ביצוע שורות בודדות.

דגש על איכות קוד (Clean Code): במימוש ה-**BulkDataReader**, נעשה שימוש במשתנה המערכת Program.Method כמקור אמת יחיד (Single Source of Truth). פרקטיקה זו מבטיחה עקביות מלאה בין הלוגיקה האפליקטיבית לבין רישום הלוגים (Logging) בבסיס הנתונים, מונעת שגיאות הקלדה ("Magic Strings") ומאפשרת תחזוקה קלה של המערכת לאורך זמן.

סיכום Workflow של הריצה

1. **שלב ה** : **Cleanup** - ניקוי תוצאות קודמות ב DB-לשמירה על אחידות ההשוואה.
2. **שלב ה** : **Pre-compilation**- קומפילציה של כל הנוסחאות לקוד מכונה לפני תחילת העיבוד.
3. **שלב ה :Evaluation**- חישוב מקבילי מסיבי על פני כל השורות בזיכרון ה.RAM-
4. **שלב ה :Bulk Insert**- הזרקת מיליון התוצאות לטבלה בתהליך Streaming מהיר.
5. **שלב ה** :**Logging**- רישום זמני הביצוע המדויקים לטבלת הלוג.

יתרונות וחסרונות(Trade-offs)

יתרונות:

- **ביצועי קצה (Native Performance):** קומפילציה בזמן ריצה (Roslyn) מאפשרת לנוסחה לרוץ במהירות של קוד מקומי, ללא הפרשנות האיטית של טקסט בכל שורה.
- **ניצול חומרה מקסימלי:** שימוש ב-Parallel Processing מאפשר לנצל את כל ליבות המעבד של השרת בו-זמנית.
- **יעילות בשינוע נתונים:** שימוש ב-SqlBulkCopy מאפשר הזרקת נתונים ישירה מהזיכרון ל-DB ללא יצירת קבצי ביניים, מה שחוסך זמן יקר ומשאבי דיסק.

חסרונות ומגבלות:

- **תקורה ראשונית (Warm-up Time):** תהליך הקומפילציה הדינמית דורש זמן עיבוד מסוים בתחילת הריצה. בעיבוד של כמות קטנה של נתונים התקורה תהיה מורגשת, אך בעיבוד מסיבי היא הופכת לזניחה.
- **צריכת זיכרון (RAM):** טעינת מיליון רשומות וביצוע חישובים במקביל דורשים משאבי זיכרון משמעותיים וניהול קפדני של ה-Garbage Collector למניעת דליפות זיכרון.

צילום חלק מההרצה של תכנית ה#C

```
[Formula 12] exp(a / 10)
  Calc: 0.007s | Load: 1.161s | Total: 1.168s

[Formula 13] power(a, 3) + (b * c)
  Calc: 0.007s | Load: 1.113s | Total: 1.120s

[Formula 14] floor(d) + 0.5
  Calc: 0.006s | Load: 1.123s | Total: 1.128s

[Formula 15] if(a > 5, b * 2, b / 2)
  Calc: 0.006s | Load: 1.106s | Total: 1.112s

[Formula 16] if(b < 10, a + 1, d - 1)
  Calc: 0.006s | Load: 1.144s | Total: 1.149s

[Formula 17] if(a == c, 1, 0)
  Calc: 0.005s | Load: 1.159s | Total: 1.164s

[Formula 18] if(d >= 50, a * b, c + d)
  Calc: 0.006s | Load: 1.114s | Total: 1.121s

[Formula 19] if(b != 0, a / b, 0)
  Calc: 0.006s | Load: 1.103s | Total: 1.109s

[Formula 20] if(a + b > 100, 100, a + b)
  Calc: 0.005s | Load: 1.084s | Total: 1.089s

[Formula 21] if(c > d, abs(c-d), abs(d-c))
  Calc: 0.006s | Load: 1.126s | Total: 1.132s


==================================================
FINAL REPORT
==================================================
  Formulas processed : 21
  Total data rows    : 1,000,000
  All calculated values are finite - results are fully comparable across methods.
==================================================

All formulas processed. Press any key to exit...
```

הסבר טכני: מימוש מערכת חישוב דינמית ב-Python

מנוע ה-Python מתמקד ביעילות תפעולית (Operational Efficiency) וב-Scalability. החישוב מבוסס על Vectorized Evaluation באמצעות ספריית NumPy, המעבירה את כובד החישוב המתמטי משכבת ה-Python האיטית לשכבת ה-C של המעבד.
כדי להתמודד עם נפחי נתונים גדולים ללא חריגה במשאבי הדיסק, הוטמע שימוש בפורמט **Apache Parquet** עם דחיסת **Snappy**. פתרון זה צמצם את נפח קבצי הביניים ב-95% (מ-GB1.5 ל-MB72 בלבד). המנוע שומר על צריכת זיכרון (RAM) קבועה ומינימלית (~MB80) ללא תלות בכמות הנוסחאות, הודות לשיטת הזרמה (Streaming) וניקוי זיכרון אגרסיבי לאחר כל שלב

1. חישוב וקטורי מבוסס NumPy & pandas.eval

לב הפתרון מתבסס על היכולת של פייתון לבצע פעולות מתמטיות על מערכים (Arrays) שלמים בבת אחת, בטכניקה המכונה Vectorization.

- **התהליך:** המערכת משתמשת בפונקציה pd.eval עם מנוע Python אופטימלי. הנוסחאות מתורגמות לביטויים שספריית NumPy מריצה ישירות על מיליון שורות בו-זמנית.
- **היתרון:** מהירות החישוב נטו (Calculation Time) היא המהירה ביותר מבין כל השיטות שנבחנו, שכן היא מתבצעת ברמת קוד C אופטימלי "מתחת למכסה המנוע" של הספריות המדעיות.

2. טכניקת Parquet + Batch Loading (אופטימיזציית אחסון ו-I/O)

במקום להשתמש בקבצי טקסט פשוטים (CSV) שהם כבדים ואיטיים, המערכת משתמשת בפורמט Apache Parquet כשלב ביניים אופטימלי לטעינה ל-SQL.

- **התהליך (Binary Storage):** התוצאות נכתבות לקובץ **Parquet** בינארי וטורי (Columnar) עם דחיסת Snappy.
- **היתרון ההנדסי:** הבחירה בפורמט **Parquet** בינארי וטורי (Columnar) היוותה מרכיב קריטי ביעילות המערכת. שיטה זו אפשרה לצמצם את נפח קובץ הביניים מנפח תיאורטי של כ-GB1.5 (בפורמט טקסטואלי רגיל) ל-MB72 בלבד – חיסכון דרמטי של כ-95% בשטח האחסון. צמצום משמעותי זה בנפח הנתונים מפחית באופן ישיר את העומס על מערכת הקבצים (Disk I/O), מזרז משמעותית את תהליכי ה-**Serialization**, ומבטיח יכולת גדילה (**Scalability**) גבוהה בטיפול בנפחי דאטה עצומים.

3. ניהול נתונים וטעינה ב-Batches

כדי להבטיח יציבות וביצועים בשלב הטעינה ל-SQL Server, הוטמעה ארכיטקטורת Batch Loading:

- **התהליך:** הקוד טוען את נתוני ה-**Parquet** חזרה לזיכרון ב"חבילות" (Chunks) ומזריק אותם ל-DB באמצעות מנגנון **fast_executemany** של **SQLAlchemy**.

- **היתרון:** גישה זו מונעת עומס יתר על זיכרון ה-RAM ומבטיחה הזרקה מהירה ויציבה, תוך שמירה על טיפוסי נתונים (Data Types) מדויקים ללא שגיאות **Parsing**.

שיקולי תכנון וניתוח חלופות – מנוע Python

במהלך פיתוח מנוע החישוב ב-Python, בוצע מחקר השוואתי בין שתי גישות ארכיטקטוניות לשינוע ואחסון הנתונים (Data Ingestion). מטרת הבדיקה הייתה לאתר את האיזון האופטימלי (The Sweet Spot) בין מהירות כתיבה למסד הנתונים לבין ניצול יעיל של משאבי מערכת (דיסק וזיכרון RAM).

1. החלופה שנבדקה ונדחתה: CSV + BULK INSERT

בשלב הראשון הוטמעה שיטה המבוססת על ייצוא תוצאות החישוב לקובץ CSV זמני וטעינתו ל-SQL Server באמצעות פקודת BULK INSERT.

ביצועים: שיטה זו הציגה זמני ריצה מהירים (כ-3.42 שניות בממוצע לנוסחה), בזכות היכולת של פקודת ה-Bulk לעקוף את שכבת ה-ORM ולהזרים נתונים ישירות לדפי הנתונים ב-SQL.

סיבות הדחייה: למרות המהירות, השיטה נפסלה בשל חסרונות קריטיים:

- **I/O Overhead:** קובץ ה-CSV הגיע לנפח חריג של כ-GB1.5 לכל ריצה, מה שיוצר עומס משמעותי על תשתית האחסון.
- **Data Integrity:** פורמט CSV (Plain Text) מחייב המרות (Parsing) בזמן הטעינה, דבר המהווה סיכון לדיוק הנתונים ודורש משאבי CPU נוספים ב-Database.
- **Scalability:** ייצור קבצי טקסט ענקיים אינו בר-קיימא במערכות ארגוניות המעבדות עשרות נוסחאות במקביל.

2. החלופה שנבחרה: Parquet + Snappy Compression

לאחר ניתוח הנתונים, הוחלט לעבור למימוש מבוסס Apache Parquet, פורמט אחסון בינארי וטורי (Columnar Storage) הנפוץ בעולמות ה-Big Data.

טבלת השוואה

פרמטר השוואה	שיטת CSV (נדחתה)	שיטת Parquet (נבחרה)
זמן ריצה ממוצע לנוסחה	~3.42 שניות	~97 שניות
נפח אחסון בדיסק	כ-1,500 MB	כ-72 MB בלבד
יעילות אחסון (Compression)	ללא	חיסכון של כ-95% (Snappy)
Data Integrity	נמוכה (Plain Text)	גבוהה (Strongly Typed)
צריכת RAM בשיא	~400 MB	~80 MB (Batch Mode)
יכולת גידול (Scalability)	מוגבלת (Risk of Disk Full)	גבוהה מאוד

ניתוח ההחלטה:

למרות הפער המשמעותי בזמני הריצה (97 שניות ב-Python לעומת ביצועי ה-C# וה-SQL), הוחלט לאמץ את פורמט ה-Parquet כחלק מהארכיטקטורה הכוללת. הסיבות לכך הן:

- **ניהול משאבי קצה (Resource Efficiency):** החיסכון של GB1.4 בכל הרצה הוא קריטי. במערכת Production, ה-Trade-off בין זמן ריצה לבין שטח אחסון נוטה לטובת ה-Parquet, המונע "ניפוח" של שרת האחסון.
- **אפיון צוואר הבקבוק (The I/O Penalty):** הניתוח הראה כי העיכוב ב-Python אינו נובע מחישובי הליבה (המבוצעים במהירות וקטורית ב-NumPy), אלא מהתקורה של סריאליזציה בינארית והזרקת נתונים דרך ה-Drivers של פייתון ל-SQL. זוהי מגבלה טכנולוגית של ה-Client שמתקבלת כחלק מדרישות ה-Robustness של המערכת.
- **דיוק פיננסי מקסימלי:** המעבר ל-Parquet מבטיח שהנתונים נשמרים בפורמט בינארי מדויק (Float/Int). בכך נמנעות שגיאות עיגול הנפוצות ב-CSV, מה שהופך את המנוע לאמין יותר לצרכים חשבונאיים.
- **Cloud-Ready Architecture:** הבחירה ב-Parquet מאפשרת אינטגרציה עתידית קלה עם שירותי ענן (כגון AWS S3 או Azure Data Lake), שם פורמט זה הוא המועדף לניתוח נתונים בהיקפים גדולים.

סיכום Workflow של הריצה

- **שלב ה-Cleanup:** ניקוי תוצאות קודמות בטבלאות ה-DB לשמירה על סביבת בדיקה נקייה ואחידה.
- **שלב ה-Vectorized Evaluation:** חישוב וקטורי של כל הנוסחאות על פני מיליון שורות בבת-אחת בתוך זיכרון ה-RAM.
- **שלב ה-Parquet Export:** ייצוא מאסיבי של מיליוני התוצאות לקובץ Parquet בינארי דחוס בנפח אופטימלי.
- **שלב ה-Batch Insert:** טעינה מהירה של הנתונים מהקובץ והזרקתם ב-Chunks לתוך טבלת התוצאות ב-SQL Server.
- **שלב ה-Logging:** רישום זמני הביצוע המדויקים לטבלת הלוג, כולל חישוב יחסי של זמן הזרקת הנתונים עבור כל נוסחה.

יתרונות וחסרונות (Trade-offs)

יתרונות:

- **עיבוד וקטורי מהיר:** שימוש בספריות NumPy ו-Pandas מאפשר לבצע חישובים מתמטיים על מיליוני שורות בבת אחת ברמת המעבד, מהירות שמתקרבת לשפות נמוכות.
- **יעילות אחסון מקסימלית:** השימוש ב-Parquet מאפשר לנהל נפחי נתונים אדירים בשטח אחסון מינימלי.
- **פשטות בפיתוח:** התחביר של פייתון מאפשר כתיבת לוגיקה מורכבת בזמן קצר, מה שהופך את המערכת לגמישה מאוד לשינויים עתידיים בנוסחאות.

חסרונות ומגבלות:

- **תקורת טעינה ל-SQL:** למרות מהירות החישוב והאחסון, שלב הטעינה מהקובץ ל-SQL Server מקומי (On-Premise) יוצר תקורה לעומת הזרמת נתונים ישירה (Streaming) הקיימת ב-C#.
- **ניהול סביבת ריצה:** המימוש דורש התקנה ותחזוקה של סביבת פייתון וספריות חיצוניות על השרת, מה שמוסיף מורכבות לתהליך ה-Deployment וה-DevOps.

מסקנה תפעולית: Python מול C#

המסקנה העולה מהמימוש היא כי Python הציגה את ביצועי החישוב הגבוהים ביותר ברמת המעבד (CPU Time) בזכות העיבוד הווקטורי של NumPy.

עם זאת, בבחינת התהליך המלא מקצה-לקצה (End-to-End), שיטת ה-C# נמצאה יעילה ומהירה יותר.

הסיבה המרכזית לכך טמונה בארכיטקטורת ה-Streaming ב-C#, המבטלת לחלוטין את הצורך בכתיבה לדיסק (אפילו בפורמט יעיל כמו Parquet) ומזרימה נתונים ישירות מהזיכרון ל-SQL.

לסיכום: Python היא הכלי האופטימלי לניתוח נתונים עמוק בזיכרון (In-Memory Analysis), בעוד ש-C# היא הפתרון המנצח לאינטגרציה מהירה ומסיבית עם בסיסי נתונים.

צילום הרצה של תוכנית Python

```
 Formula 15: 0.0126s  →  formula_015.parquet
 Formula 16: 0.0125s  →  formula_016.parquet
 Formula 17: 0.0060s  →  formula_017.parquet
 Formula 18: 0.0130s  →  formula_018.parquet
 Formula 19: 0.0091s  →  formula_019.parquet
 Formula 20: 0.0160s  →  formula_020.parquet
 Formula 21: 0.0190s  →  formula_021.parquet

 Parquet total: 233.5 MB  (vs ~561 MB for CSV)

tep 3: Loading results into SQL Server...
 Inserted formula_001.parquet
 Inserted formula_002.parquet
 Inserted formula_003.parquet
 Inserted formula_004.parquet
 Inserted formula_005.parquet
 Inserted formula_006.parquet
 Inserted formula_007.parquet
 Inserted formula_008.parquet
 Inserted formula_009.parquet
 Inserted formula_010.parquet
 Inserted formula_011.parquet
 Inserted formula_012.parquet
 Inserted formula_013.parquet
 Inserted formula_014.parquet
 Inserted formula_015.parquet
 Inserted formula_016.parquet
 Inserted formula_017.parquet
 Inserted formula_018.parquet
 Inserted formula_019.parquet
 Inserted formula_020.parquet
 Inserted formula_021.parquet

 Insert done in 2161.82s

=====================================================
INAL PERFORMANCE REPORT (PYTHON)
=====================================================
 Formulas evaluated : 21 / 21
 Total rows inserted: 21,000,000
 Calculation time   : 0.27s
```

הסבר טכני: מימוש באמצעות SQL Dynamic (Stored Procedure)

המימוש ב-T-SQL בוצע באמצעות פרוצדורה דינמית המייצרת שאילתות INSERT...SELECT בזמן אמת. יתרונה המרכזי של שיטה זו הוא ביטול הצורך בשינוע נתונים (Data Movement) מחוץ לבסיס הנתונים.

לוגיקת המרה: המנוע כולל רכיב 'נרמול' המתרגם נוסחאות לוגיות (כגון if) למבני CASE WHEN תקניים של SQL.
למרות היותו מנוע סינכרוני (Single-threaded), הוא מהווה חלופה יציבה ופשוטה לתחזוקה עבור מערכות שבהן לא קיימת תשתית אפליקטיבית חיצונית.

פירוט שיטות העבודה בקוד (Methods & Strategies)

- **שימוש ב-Dynamic SQL ו-sp_executesql:**
היות והנוסחאות נשמרות כטקסט בטבלה, לא ניתן להריץ אותן כשאילתה רגילה. הקוד משתמש ב-Dynamic SQL כדי לבנות מחרוזת פקודה בזמן ריצה ולהוציא אותה לפועל. השימוש ב-**sp_executesql** (ולא ב-EXEC פשוט) מאפשר שימוש בפרמטרים, מה שמשפר את הביצועים בזכות שימוש חוזר בתוכנית ההרצה (Plan Reusability) ומגן על המערכת מפני SQL Injection.
- **מנגנון תרגום נוסחאות (Formula Parsing):**
הפרוצדורה כוללת לוגיקה לניתוח טקסטואלי של הנוסחאות לצורך התאמתן לתחביר השרת:
  - **נרמול אופרטורים:** המרת אופרטורים של שפות תכנות (כמו == או !=) לאופרטורים חוקיים של SQL (= ו-<>).
  - **חילוץ ביטויים מותנים:** המערכת מזהה מבנה של פונקציית if(cond, then, else), מחלצת את החלקים הרלוונטיים באמצעות פונקציות טקסט (CHARINDEX, SUBSTRING), ובונה מהם מבנה של CASE WHEN – המבנה הלוגי המקביל ב-SQL המאפשר חישוב מותנה ברמת השורה.
- **אופטימיזציית קריאה (NOLOCK Hint):**
בעת עיבוד מיליון השורות מטבלת המקור, הקוד משתמש ברמז (Hint) מסוג WITH (**NOLOCK**). זהו כלי קריטי ב-High Performance SQL המאפשר לקרוא נתונים מבלי להמתין לנעילות (Locks) של תהליכים אחרים, מה שמאיץ משמעותית את מהירות השליפה והעיבוד.
ניתוח מקצועי (Trade-off): חשוב לציין כי השימוש ב-NOLOCK נבחר כאן מאחר שטבלת המקור היא סטטית (Read-only) במהלך הבנצ'מרק. במערכת פיננסית חיה (Production), שבה מתבצעות כתיבות במקביל, שימוש כזה עלול להוביל לקריאת נתונים לא עקביים ("Dirty Reads"). בסביבת אמת, ההמלצה ההנדסית היא להשתמש ב-READ_COMMITTED_SNAPSHOT ברמת בסיס הנתונים, המאפשר קריאות מהירות ולא חוסמות תוך שמירה על אמינות נתונים מלאה.

סיכום Workflow של הריצה (SQL)

1. **שלב ניקוי הנתונים:** מחיקת תוצאות קודמות המשויכות לשיטת ה-SQL_DB בלבד, כדי לאפשר השוואה נקייה מול שיטות אחרות מבלי לפגוע בנתונים שלהן.
2. **שלב הסריקה (Cursor):** שימוש ב-CURSOR מסוג FAST_FORWARD (המהיר ביותר ב-SQL Server). הקורסור משמש רק כדי לעבור על רשימת הנוסחאות (Iterative), בעוד שהחישוב עצמו על מיליון השורות מתבצע בבת-אחת (Set-based operation).

3. **שלב הבנייה הדינמית:** עבור כל נוסחה, הפרוצדורה בונה פקודת INSERT INTO ... SELECT הכוללת את הנוסחה המתורגמת.
4. **שלב הביצוע והזרקה:** המנוע מריץ את השאילתה המורכבת. החישוב וההזרקה לטבלת התוצאות מתבצעים כפעולה אחת אטומית, מה שמונע "טיול" מיותר של הנתונים לזיכרון ה-RAM החיצוני.
5. **שלב המדידה:** שימוש ב-DATEDIFF_BIG עם דיוק של מיקרו-שניות (Microseconds) לקבלת מדידת זמן מדויקת ככל הניתן של ביצוע החישוב בתוך השרת.

יתרונות וחסרונות (Trade-offs)

יתרונות:

- **אפס שינוע נתונים (Zero Data Movement):** הנתונים אינם עוזבים את השרת, ולכן אין עומס על הרשת (Network Latency) ואין צורך בזיכרון RAM יקר באפליקציה החיצונית. זוהי השיטה היציבה ביותר מבחינת תשתית.
- **אבטחה וניהול:** ניצול מנגנוני ההרשאות המובנים של ה-DB וביצוע אופטימיזציה של תוכניות הרצה (Execution Plans) דרך המנוע של SQL Server.

חסרונות ומגבלות:

- **עומס על משאבי המערכת (CPU & Memory):** הרצה מאסיבית של נוסחאות בתוך ה-DB גוזלת משאבי מעבד וזיכרון ישירות משרת הנתונים. בסביבת אמת (Production), הדבר עלול להוביל להאטה משמעותית בשאילתות של משתמשים אחרים, ניצול יתר של ה-Transaction Log ואף לנעילת טבלאות.
- **יעילות חישובית:** מנוע ה-SQL Server פחות יעיל בביצוע חישובים מתמטיים מורכבים (כמו פונקציות טריגונומטריות או לוגריתמים) בהשוואה לחישוב Native ב-C# או עיבוד וקטורי ב-Python.
- **קושי ב-Debugging ותחזוקה:** איתור טעויות לוגיות או שגיאות חישוב בתוך שאילתות SQL דינמיות מורכב משמעותית לעומת סביבות פיתוח מודרניות. חוסר היכולת לבצע Step-by-step debugging הופך את תחזוקת הקוד למאתגרת עבור הצוות הטכני.

אבטחת מידע וניהול סיכונים (Security & Code Integrity)

בעיבוד של נוסחאות דינמיות המגיעות ממקור חיצוני, קיימת חשיבות עליונה למניעת סיכוני הזרקת קוד (Injection). המערכת תוכננה תחת עקרון ה-Sandboxing, המבטיח בידוד מוחלט של הרצת הנוסחאות:

- **מניעת SQL Injection:** במימוש ה-SQL נעשה שימוש ב-sp_executesql המבצע הפרדה מוחלטת בין פקודות המנוע לבין נתוני הנוסחה (Parameterization), ובכך חוסם כל ניסיון להרצת פקודות זדוניות דרך שדות הטקסט.
- **בידוד סביבת הריצה (C# & Python):** במימוש ה-C# נעשה שימוש ב-Roslyn בסביבה מוגדרת מראש המגבילה את הנוסחה לחישובים מתמטיים בלבד ומונעת גישה למשאבי מערכת ההפעלה.

בדומה לכך, במימוש הפייתון הוגבלה הערכת הביטויים (Evaluation) למילון פונקציות סגור (Safe Local Dictionary), מה שמבטל את הסיכון להרצת קוד זדוני (RCE) ומבטיח את יציבות השרת.

סיכום ממצאים והמלצה על השיטה הנבחרת

לאחר בחינת שלוש שיטות המימוש (SQL, C#, Python) על מסד נתונים של מיליון רשומות ו-21 נוסחאות שונות, להלן סיכום הביצועים והמלצתי המקצועית.

1. ריכוז נתוני ביצועים (ממוצע לנוסחה בשניות)

על פי דוח הלוגים הסופי (טבלת t_log), אלו זמני הריצה הממוצעים שנתקבלו:

שיטת מימוש	זמן ממוצע לנוסחה (שניות)	זמן כולל ל-21נוסחאות (שניות)	יתרון מרכזי
C# (Compiled & Parallel)	~1.28s	~26.8s	ביצועי Native והזרקה ישירה
SQL Dynamic	~3.14s	~65.9s	פשטות תחזוקה ללא שרת אפליקציה
Python (Pandas)	~3.42s	~71.8s	יכולות In-Memory Analytics

וידוא שלמות ודיוק הנתונים (Data Integrity & Validation)

מעבר לבחינת המהירות, בוצע תהליך ולידציה קפדני בין שלוש השיטות.
השוויתי את התוצאות המתמטיות של עשרות אלפי רשומות מתוך טבלת t_results-ה בין ה-C#, ה-Python וה-SQL. הממצאים הראו זהות מוחלטת בתוצאות (Precision), מה שמוכיח כי לוגיקת התרגום (Parsing) בכל אחת מהשפות תקינה ושומרת על רמת דיוק אחידה ללא תלות בסביבת ההרצה.

2. ניתוח תוצאות והשוואת מתודולוגיות

במסגרת המבדק, נבחנו שלוש גישות שונות לעיבוד מאסיבי של מיליון שורות נתונים. להלן דירוג היעילות והניתוח ההנדסי לכל שיטה:

עדיפות ראשונה: C# – The High-Performance Engine

ניתוח הנדסי:

שיטת ה-C# הוכיחה עליונות מוחלטת עם זמן הביצוע הנמוך ביותר וצריכת משאבים אופטימלית.

השילוב בין קומפילציית נוסחאות לקוד מכונה (Native IL) באמצעות Roslyn, לבין עיבוד מקבילי מסיבי (Multi-threading), אפשר למעבד לבצע חישובים מורכבים במילישניות.

יתרון ה-Zero-Copy: השימוש במנגנון ה-SqlBulkCopy בשילוב Custom IDataReader אפשר הזרמת נתונים ישירה ל-SQL ללא יצירת קבצי ביניים וללא העמסת ה-RAM באובייקטים כבדים. זוהי השיטה היציבה ביותר לנפחי נתונים הולכים וגדלים.

עדיפות שנייה: SQL Dynamic – The In-Place Processor

השיטה הישירה ב-SQL סיפקה תוצאות יציבות (כ-3.1 שניות לנוסחה) ומהווה פתרון פשוט לתחזוקה.

ניתוח הנדסי:

יתרונה המרכזי הוא ביטול הצורך בשינוע נתונים מחוץ ל-DB. עם זאת, היא "נועלת" משאבי בסיס נתונים קריטיים בזמן החישוב ומוגבלת ביכולות העיבוד המקבילי שלה לעומת שרת אפליקציה ייעודי.

מתי להשתמש? במערכות ללא תשתית אפליקטיבית חיצונית שבהן פשטות הקוד קודמת למהירות החישוב.

עדיפות שלישית: Python – The Big Data Specialist

למרות שפייתון דורגה שלישית במבחן ה-End-to-End, היא הציגה יעילות מרשימה ברמת חישובי המעבד (CPU Time) בזכות טכניקת ה-Vectorization.

צוואר הבקבוק: עיקר העיכוב נבע מפעולות ה-I/O (כתיבה לקובץ וייבוא ל-SQL). למרות השימוש בפורמט Parquet שצמצם את נפח האחסון ב-95%, המעבר דרך שכבת ביניים מהווה תקורה (Overhead) לעומת הזרמה ישירה.

מתי להשתמש? זוהי השיטה המועדפת לתרחישי Data Science שבהם הנתונים נשארים בזיכרון (In-Memory) או ב-Data Lakes ללא צורך בהזרקה חוזרת לבסיס נתונים רלציוני.

4. סיכום ומסקנות: ההמלצה להטמעה

לאחר ניתוח מעמיק של ביצועי המערכת ותחת דרישות המבדק לעיבוד מהיר ויציב של מיליון שורות, המלצתי היא להטמיע את מנוע ה-C# כפתרון הליבה, מהסיבות הבאות:

- **יציבות תחת עומס (System Robustness)**: בזכות ניהול זיכרון דטרמיניסטי (Strong Typing) ושימוש ב-Streaming, המערכת חסינה לקריסות זיכרון גם בנפחי ענק.
- **Native Performance:** קומפילציה בזמן ריצה המנצלת את כל ליבות המעבד במינימום תקורה ניהולית.
- **Zero-Overhead Data Flow:** הזרקה ישירה ללא יצירת קבצי ביניים, מה שחוסך זמן יקר ושטח אחסון בשרת.
- **Production Readiness:** פתרון יציב, קל לניטור (Logging) ובעל יכולת טיפול בשגיאות חישוב בזמן אמת בצורה המקצועית ביותר.